Design and Implementation of a RISC-V Processor with Custom SIMD Instructions

CE-362: Computer Architecture Projects

Prof. David Zeretsky

March 14th, 2025

Mark Chauhan, Xiaoyi Lu, Ziad Elbadry Shaker

Northwestern | McCORMICK SCHOOL OF ENGINEERING

Table of Contents

Executive Summary 3
Introduction 4
Broader Considerations 4
Design Constraints and Requirements 5
Design Description 6
Design Optimizations 9
Engineering Standard 10
Testing/Simulation 10
FPGA Implementation/Synthesis 13
Ethical Considerations 14
Conclusions 14
References 15

Table of Figures

Figure 1: Encoding Table 7
Figure 2: System Architecture 8
Figure 3: Waveforms 10
Figure 4: Execution Time 11
Figure 5: Matrix Multiplication 11
Figure 6: FPGA Implementation 13

Executive Summary

Problem:
Modern computational tasks, such as machine learning and signal processing, require efficient handling of parallel data operations. Traditional RISC-V processors are limited in performing these tasks efficiently due to their scalar nature. This project addresses this limitation by designing and implementing a RISC-V processor with custom SIMD (Single Instruction, Multiple Data) instructions to enable efficient parallel execution of vector-based computations.

Purpose and Scope:
The objective was to develop a RISC-V processor with custom SIMD instructions to improve the performance of parallel data operations. The processor is designed for integer-based computations on an FPGA platform, excluding floating-point operations to simplify the architecture. It includes vector registers to store and manage data for parallel execution, eliminating the need for complex memory access patterns.

Methodology:
The processor design was implemented in Verilog and simulated using Icarus Verilog (Iverilog). GTKWave was used to visualize and debug waveforms. Custom SIMD instructions for dot product, multiplication, division, addition, and subtraction were integrated into the pipeline. An automated Python script was developed to generate test cases, run simulations, and analyze results. The processor was synthesized and tested on an FPGA, with iterative improvements based on benchmarking results to refine execution timing and resource utilization.

Design and Benefits:
The processor features a streamlined pipeline and a custom instruction set, providing several benefits:

- **Dot Product:** Computes the dot product of two vectors in a single cycle.
- **Arithmetic Operations:** SIMD-based multiplication, division, addition, and subtraction enable efficient vector processing.
- **Vector Registers:** Data is stored in vector registers for quick access and parallel execution.
- **Efficient Pipeline:** The pipelined design increases throughput by supporting concurrent execution.
- **Testing and Verification:** Automated testing and waveform analysis ensured accuracy and stability.
- **Scalability:** Compact and resource-efficient, making it suitable for FPGA deployment and future expansion.

Benchmarking shows up to a 5x performance improvement over scalar processing for vector-based computations, highlighting the efficiency of the custom SIMD implementation.

Introduction

The goal of the project was to design and implement RISC-V vector instructions utilized for matrix multiplication, allowing for efficient computation on FPGA hardware. Scalar implementations are inefficient and can get overly complicated for the user. This report explains the group's design process, testing, challenges, and final implementation.

The approach for the design started with creating vector instructions that can be used/useful for matrix multiplication. Next the group implemented these instructions into a single-cycle processor by mid-quarter. This processor was then extensively tested through simulation and verification explained later. Then implemented on an FPGA to verify usability and then was translated into a pipelined processor that had its own testing and implementation on the FPGA. Finally some optimization and documentation was done to complete the project.

Broader Considerations

Some considerations of this processor are applications in Artificial Intelligence and Machine Learning methods that use several matrix multiplication calculators. Robotics movement and control rely significantly on matrices as well. To adhere with the RISC-V standard, allowing for easy accessibility.

Design Constraints and Requirements

This project focuses on FPGA implementation featuring the RV32I instruction set architecture with SIMD capabilities and dual-core. The key innovation in this design is the sharing of ALUs across different cores to efficiently execute SIMD instructions. The processor architecture aims to balance general-purpose computing and parallel data processing.

Key features of this project include:

- Instruction Set: Supporting the RISC-V RV32I ISA with additional SIMD instructions
- Custom SIMD Instructions: Implementing vector operations such as multiplication, subtraction, addition, and division
- Processor Design: Transitioning from single-cycle to pipelined architecture, then extending to a dual-core system
- Memory Hierarchy: Implementing instruction and data caches to improve execution efficiency
- Verification Methods: Verifying the design through waveform analysis, execution time, and RTL synthesis

Design Description

The processor follows a five-stage pipeline structure: Instruction Fetch, Instruction Decode, Execute, Memory Access, Write Back. The addition was in two separate ALUs and register files, one for scalar operations/values and vector operations/values. The instructions are fixed at 32 bits to maintain consistency with the RISC-V specifications. The separate registers would each have 32 for scalar and vectors for each. The scalar registers would be 32-bit wide each consistent with standard RISC-V specifications, and the vector registers are 128-bit wide each to hold four 32-bit elements for the SIMD operations. The ALUs in the execution unit handle the scalar and vector operations. ALU will handle standard operations and ALUV handles SIMD operations for parallel vector process, addition, subtraction, multiplication, division, and dot product. The process supports specific-SIMD operations to execute certain programs like matrix multiplication.

Custom SIMD Instructions

31 25	24 20	19 15	14 12	11 7	6 0	
funct7	**rs2**	**rs1**	**funct3**	**rd**	**opcode**	
0	rs2	rs1	elem[1:0]	rd	opcode	V1
0	rs2	rs1	funct3	rd	opcode	V2
imm[11:0]		0	elem [1:0]	rd	opcode	V3

Instruction	FMT	Opcode	funct3	Description
vdot	V1	1000011		rd[elem] = rs1 . rs2
vadd	V2	1001100	0x0	rd = rs1 + rs2
vmul	V2	1001100	0x1	rd = rs1 * rs2
vsub	V2	1001100	0x2	rd = rs1 - rs2
vdiv	V2	1001100	0x3	rd = rs1 / rs2
vli	V3	1001111		rd[elem] = imm

Figure 1: Encoding table of the custom vector instructions, provides how the instructions are translated into opcode for the processor

Pipeline Architecture The architecture consists of a five-stage pipeline(Instruction Fetch, Decode, Execute, Memory Access, and Write Back), with a shared ALU for SIMD Operations.

Along with a separate register file and ALU for vector instructions.

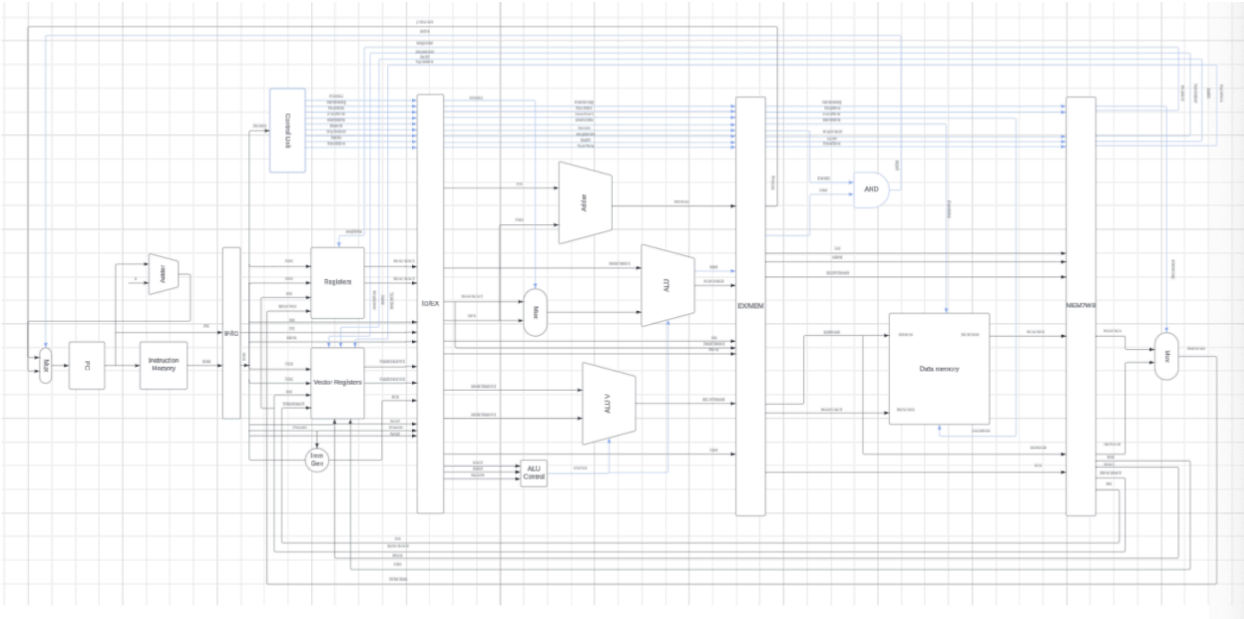

Figure 2: The system architecture of the project displays the five stages of the pipeline, and how the double registers and ALUs function overall.

[System Architecture]

Design Optimizations

To improve the performance and efficiency of our RISC-V processor with custom SIMD instructions, we applied several key optimizations:

1. **Pipelining:** Implemented a fully pipelined architecture to increase instruction throughput by allowing concurrent execution of multiple instructions, reducing latency and improving performance.

2. **Vector Registers and Reuse:** Introduced vector registers to store data for parallel execution, minimizing memory access delays and enabling fast data reuse.

3. **Custom SIMD Execution Unit:** Designed a dedicated SIMD execution unit that performs all SIMD instructions (dot product, multiplication, division, addition, and subtraction) in a single cycle, reducing execution time and improving efficiency.

These optimizations resulted in a 5x performance improvement over scalar execution.

Engineering Standards

Three engineering standards were held, RISC-V ISA, IEEE 1800, IEEE 1800.2. RISC-V ISA ensures interoperability with other RISC-V architectures. IEEE 1800 is the guidelines for SystemVerilog-based design for industry-wide compatibility. IEEE 1800.2 verification follows Universal Verification Methodology for replication and structured testing.

Testing/Simulation

We employed multiple methods to verify the correctness and performance of our RISC-V processor with custom SIMD instructions:

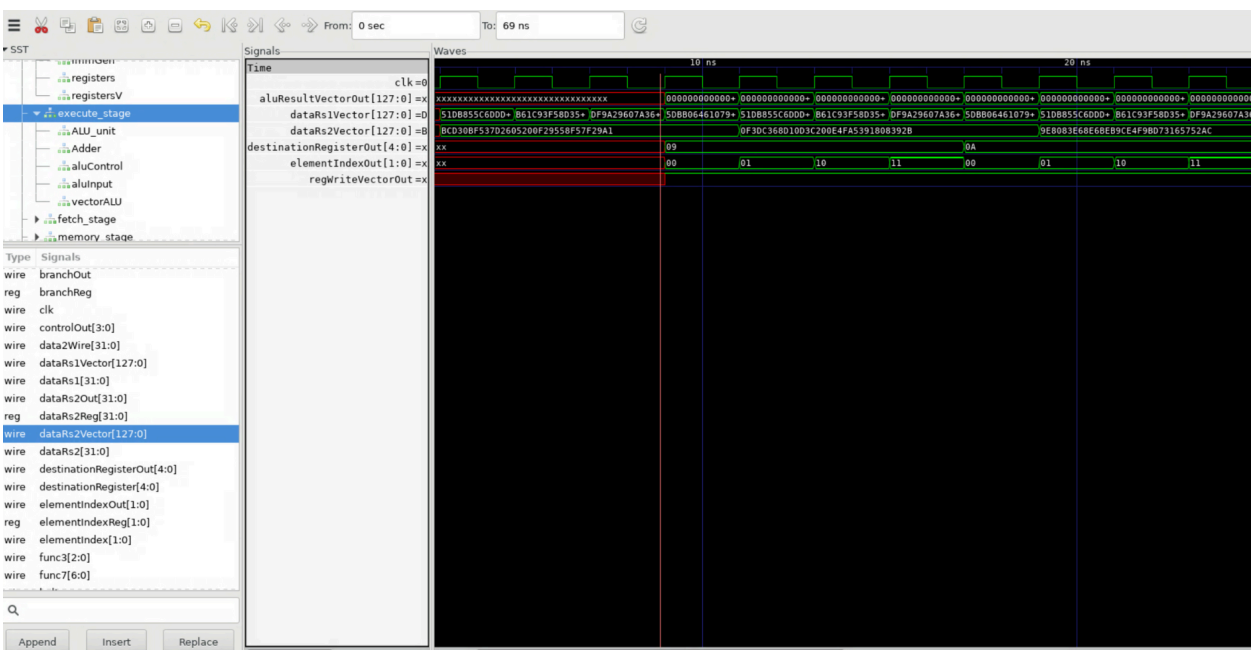

Figure 3: Waveforms displaying clock cycle, vector register values and successful matrix multiplication

```
inal  Help                                  RISC-Processor [SSH: moore.wot.eecs.northw
tb.v  U   test_0_output.log U ×   mem_in.hex U   regs_in.hex U
Pipeline > output_log > test_0_output.log
1  WARNING: tb.v:54: $readmemh(mem_in/test1_mem.hex): Not enough w
2  WARNING: tb.v:56: $readmemh(regs_in/test1_regs_inv.hex): Not en
3  Loading instructions from
4  VCD info: dumpfile pipeline.vcd opened for output.
5  Execution Time:         69
6
```

Figure 4: Execution time displayed after Verilog tests

```
program 2: 3x3 matrix multiplication
matrix C :
[[-1303188702  1001800981  1664613129]
 [-1851745198 -1108625793  -990057619]
 [ 1018934284   782980778  -255929256]]
matrix D :
[[-1596481993  2083276858 -1680208537]
 [   18804461  2073581062 -1831566109]
 [ 1798904204 -1814223544   528662279]]
expected output matrix :
[[ 5093835179492713643 -3657567652672345670  1234782228090257036]
 [ 1154411940721826765 -4360337524809178314  4618433402848551062]
 [-2072378919497689578  4210609208684429172 -3281403283341769734]]
program 3: 2x2 convolution
matrix E :
[[ -903023704 -1506476530]
 [ 1198596380    63324597]]
kernel :
[[ 2  0]
 [-2  2]]
expected output :
-4076590974
regs_out:  regs_out/test99_regs_outv.hex
✅ Test 99 PASSED
```

Figure 5: Displayed 3x3 matrices, multiplied to show a successful matrix multiplication process

1. **RTL Synthesis:** The processor was synthesized using FPGA tools to ensure that the hardware description accurately reflects the intended logic.
2. **Waveform Analysis:** We used GTKWave to observe signal transitions and verify instruction execution. As shown in *Figure 3*, the waveform for a 4×4 matrix multiplication confirms that SIMD instructions are correctly executed in the pipeline. The SIMD execution unit completes the actual computation (dot product, multiplication, division, addition, subtraction) within a single cycle in the execution stage, ensuring high throughput and low latency.
3. **Execution Time Calculation:** We measured performance by analyzing cycle counts during the execution of various workloads. *Figure 4* shows the execution times for 4×4 matrix multiplication, 3×3 matrix multiplication, and 2×2 convolution, confirming that the SIMD execution unit reduces computation time compared to scalar processing.
4. **Automated Testing:** A **Python script** was developed to automate test generation, execution, and result comparison. The script executed tests using Icarus Verilog (Iverilog) and compared outputs against expected results from numpy. The simulation output for a 4×4 matrix multiplication is shown in *Figure 5*, demonstrating that the computed results match expected values, confirming the correctness of the SIMD execution.

The simulation outputs verified that the SIMD execution unit correctly handled all operations (dot product, multiplication, division, addition, subtraction) within a single cycle in the execution stage. Benchmark results showed consistent performance improvements, with up to a 5x reduction in execution time over scalar processing.

FPGA Implementation/Synthesis

FPGA implementation is critical to verify if the hardware design works in the real situation. Our implementation not only includes a program of a single test case, but utilizes the switches, LCD and LED segments on the FPGA board as well. To visualize the values of the vector register. The 16x2 letters LCD screen is used to display 32 hex values which is exactly the size of the 128 bits vector register. Due to multiple vector registers, the switches are employed to control the address of the vector register displayed on the LCD screen. The LED segments are used to display the values of switches, also the address of the vector registers. In the meanwhile, to simplify the configuration, although the maximum available frequency of the design is higher, the built-in 50Mhz clock is used as the main clock. The implementation verified that our hardware design works on the FPGA board.

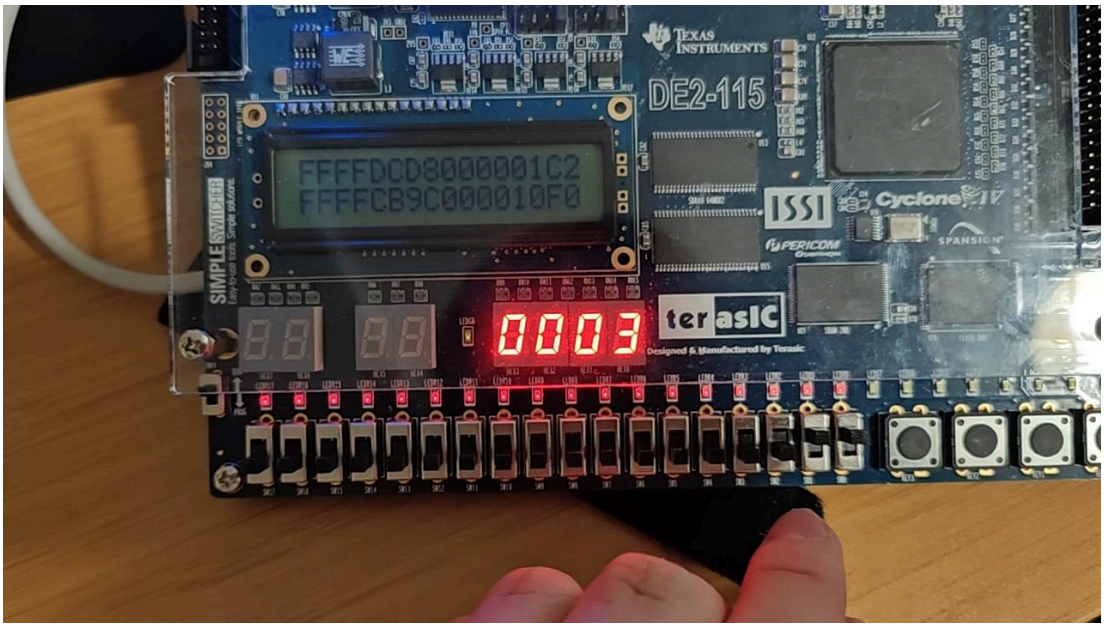

Figure 6: This image displays FPGA results of the processor, which parallel what has been produced in iverilog in software testing.

Ethical Considerations

The pipelined design seeks to minimize power consumption and allow for energy efficiency. Reliability and safety to allow for built-in fault tolerance mechanisms to prevent errors.

Conclusion and Future Work

This project is able to provide an implementation of a RISC-V processor with SIMD capabilities with improvements in execution speed and efficiency. By utilizing custom vector instructions and a pipelined architecture, the design leads to reduced execution cycles and times for matrix computations. The validation process through simulation and FPGA implementation confirmed accuracy and the performance improvements. The result showed that custom SIMD support in the RISC-V process is beneficial in execution matrix multiplication.

In terms of future work, a significant goal could be a block ram and memory management. The block ram enables a larger temporary storage, and memory and register values could be pointer-based to a larger memory storage to allow for larger values. A variable length SIMD value/varied matrix length can be implemented if the group worked on adjusting the opcode and enhancing the memory. In order to further increase the speed, dual-cores can be implemented for the scalar and vector operations for further speed.

References

Patterson, D. A., & Hennessy, J. L. (2017). *Computer organization and design: The hardware/software interface (RISC-V edition)*. Morgan Kaufmann.

RISC-V International. (2021). *RISC-V "V" vector extension: Version 1.0*. Retrieved from https://riscv.org